Monte Carlo Integration

Mark M. Fredrickson (mfredric@umich.edu)

Module 1: Integrate

Computational Methods in Statistics and Data Science (DATASCI 406)

Integrals as Expectations

Expectation

Suppose we are going to compute $g(X)$ for a random variable X.

We often want to "average over" X to get a sense of a typical value for $g(X)$. We define the **expectation** of $g(X)$ as:

$$E(g(X)) = \sum_{i=-\infty}^{\infty} P(X = x)g(x) \qquad \text{(discrete)}$$

$$E(g(X)) = \int_{-\infty}^{\infty} g(x)f(x)\,dx \qquad \text{(continuous)}$$

We call the **expectation of the identity function**, $E(X)$ the **mean of** X.

We call $E\left[(X - E(X))^2\right]$ the **variance**.

Expectation and Inference

Recall, a **parameter** is an **unknown quantity** in a statistical model.

We previously discussed two forms of **statistical inference** for parameters:

- **Estimation**: making informed guesses about population values.
- **Testing**: checking if the data conform to a specific value of the parameter.

Expectations are useful for both:

- Important **operating characteristics** of estimators (e.g. bias)
- Computing **Type I and Type II** error rates of tests
- Many parameters can be **expressed as expectations**.

Monte Carlo Integration

Recall your introduction performing integrals using **Riemann sums**:

$$\int_a^b h(x)\,dx \approx \sum_{i=0}^{n} h(a + di/2) \times d, \quad d = \frac{b-a}{n}$$

(or using the trapezoid rule or any other similar technique).

Straightforward, but

- How do you pick d? Alternatively, if spacing is unequal, how do you pick the regions?
- When integrating in k-dimensions we need to take n^k samples (gets big fast!)

Solution: let $h(x)$ help us pick the most important regions and integrate using randomly selected points.

Example: Universal function integrator

Suppose we want to evaluate a complex function over the region 0 to 1 (for a math class).

For example,

$$\int_0^1 \left(\log\left(\frac{1}{x}\right)\right)^3 dx$$

This integral doesn't have a **closed form solution**, so our usual techniques do not work. (BTW: This is the $\Gamma(4)$ function.)

Example continued

Recall: if $U \sim U(0,1)$ then the density function is $f(u) = 1$.

$$\int_0^1 \left(\log\left(\frac{1}{x}\right)\right)^3 dx = \int_0^1 \left(\log\left(\frac{1}{x}\right)\right)^3 f(x)\, dx = E\left(g(U)\right)$$

where

$$g(X) = \left(\log\left(\frac{1}{x}\right)\right)^3$$

Conveniently, computers are great at generating lots of $U(0,1)$ random variables!

We'll **approximate** $E(g(U))$ with **the sample mean of** $g(U_i)$.

Monte Carlo Gamma Function

We **estimate the integral** using draws from $U(0, 1)$ and the sample mean of the function values:

```
g <- function(u) { log(1/u)^3 }
mean(g(runif(1000000)))
```

```
## [1] 6.014
```

Since the exponent was integer, $\Gamma(a) = (a - 1)!$, in this case **3! = 6**.

Monte Carlo integration: the general case.

We wish to estimate $\int_a^b g(x)\,dx$. Suppose X is a random variable with:

- Support $[a, b]$ (where either $a \to -\infty$ or $b \to \infty$)
- Density $f(x)$ that is non-zero over $[a, b]$

then

$$\int_a^b g(x)\,dx = \int_a^b g(x)\frac{f(x)}{f(x)}\,dx = \int_a^b \frac{g(x)}{f(x)} f(x)\,dx = \mathrm{E}\left(h(X)\right)$$

where $h(x) = g(x)/f(x)$.

Sample from X i.i.d. and estimate using $(1/n)\sum_{i=1}^n g(X_i)/f(X_i)$.

Example: Integral of $1/2^x$

Suppose we need to compute

$$\int_0^\infty \frac{1}{2^x}\, dx$$

What kind of distribution has the support $[0, \infty)$?

The **exponential distribution** has

- Support $[0, \infty)$
- Density function $f(x) = \exp\{-x\}$ (keeping the usual parameter $\lambda = 1$)
- Random number generator `rexp`

Example continued

We've identified a variable on $[0, \infty)$, what expectation should we estimate?

Let $g(x) = 1/2^x$.

$$\int_0^\infty g(x)\, dx = \int_0^\infty \frac{g(x)}{\exp\{-x\}} \exp\{-x\}\, dx = \mathrm{E}\left(h(X)\right)$$

Implementing

```
g <- function(x) {
  1/(2^x)
}
h <- function(x) { g(x) / dexp(x) } ## R's exp. density function
```

```
k <- 100000
hX <- h(rexp(k))
mean(hX)
```

```
## [1] 1.442
```

```
1/log(2)
```

```
## [1] 1.443
```

Distributions in R

R has a naming convention for functions related to **distributions** based on **prefixes**:

- `r`: generated random values from the distribution (deviates)
- `d`: evaluates the probability density or mass function at its argument
- `p`: implements the CDF for $P(X \leq x)$
- `q`: implements the quantile function (inverse of CDF) and finds x such that $P(X \leq x) = p$.

See `?Distributions` for a list of built in distributions with these functions.

Monte Carlo: Special case

Sometimes the quantity we want **is already an expectation**:

$$E(g(X)) = \int g(x) f(x)\, dx$$

for a random variable X we know how to sample. Then there is **no trick at all**: sample from X and average $g(X_i)$.

Example: $X \sim \text{Exp}(1)$, estimate $E(\log(1 + X))$:

```
mean(log(1 + rexp(100000)))
```

```
## [1] 0.5944
```

The density f never appears in the code — it is baked into `rexp`.

Why It Works

Estimating a mean (in general)

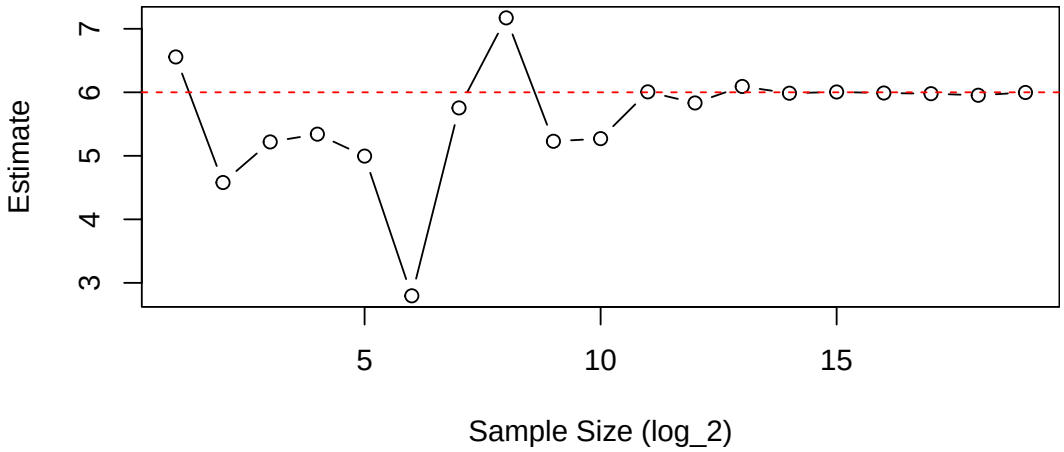
Why does this work? Suppose that $X_i \overset{\text{iid}}{\sim} F$ and $E(g(X)) < \infty$. The **sample mean of** $g(X_i)$ is **unbiased** for the population quantity $E(g(X))$.

$$\begin{aligned} E\left(\frac{1}{n}\sum_{i=1}^{n} g(X_i)\right) &= \frac{1}{n}\sum_{i=1}^{n} E(g(X_i)) \\ &= \frac{1}{n} nE\left(g(X_1)\right) \\ &= E(g(X)) \end{aligned}$$

Provided we can sample from F, **the sample mean is a good estimator of** $E(g(X))$.

Larger sample → better estimates

```
g <- function(u) { log(1/u)^3 }
ms <- map_dbl(2:20, ~ mean(g(runif(2^.x))))
```



Convergence of Estimates

When $\mathrm{E}\left(g(X)\right) < \infty, \mathrm{E}\left(g(X)^2\right) < \infty$ and X_i are IID, by the **weak law of large numbers**,

$$\hat{\theta}_n = \frac{1}{n}\sum_{i=1}^{n} g(X_i) \xrightarrow{P} \mathrm{E}\left(g(X)\right) = \theta$$

as $n \to \infty$.

Reminder: The notation $\xrightarrow{P}$ indicates "convergence in probability":

$$\hat{\theta}_n \xrightarrow{P} \theta \equiv \lim_{n\to\infty} \Pr(|\hat{\theta}_n - \theta| < \epsilon) = 1, \quad \text{for any } \epsilon > 0$$

In words: we can draw a large enough sample to concentrate all the probability of $\hat{\theta}_n$ within $\pm\epsilon$ of θ.

Uncertainty in estimating $\mathsf{E}\,(g(X))$

Under these same conditions ($X_i \overset{\text{iid}}{\sim} F, \mathsf{E}\,(g(X)) < \infty, \mathsf{E}\,(g(X)^2) < \infty$), the **central limit theorem** states:

$$\frac{\hat{\theta}_n - \theta}{\hat{\mathsf{s}}_n/\sqrt{n}} \xrightarrow{D} N(0,1) \quad \text{(convergence in distribution)}$$

where:

$$\hat{\theta}_n = \frac{1}{n}\sum_{i=1}^{n} g(X_i), \quad \hat{\mathsf{s}}_n = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(g(X_i) - \hat{\theta}_n)^2}$$

Confidence Intervals for θ

Our Monte Carlo answer is an **estimate**, not an exact solution, so we report a **confidence interval** (a plausible range) for the value of the integral.

By the CLT,

$$\hat{\theta}_n \approx N(\theta, \tau^2/n), \quad \tau^2 = \mathrm{Var}(g(X))$$

an approximate $100 \times (1-\alpha)\%$ **confidence interval for** θ is given by:

$$\hat{\theta} \pm t_{\alpha/2}(n-1) \times \hat{s}/\sqrt{n}$$

where $t_{\alpha/2}(n-1)$ is the $\alpha/2$ quantile of Student's t-distribution with $n-1$ degrees of freedom.

Confidence Intervals in R

To estimate

$$\int_0^\infty \frac{1}{2^x}\, dx$$

we used an exponential random variable so generate the estimate:

```
mean(hX)
```

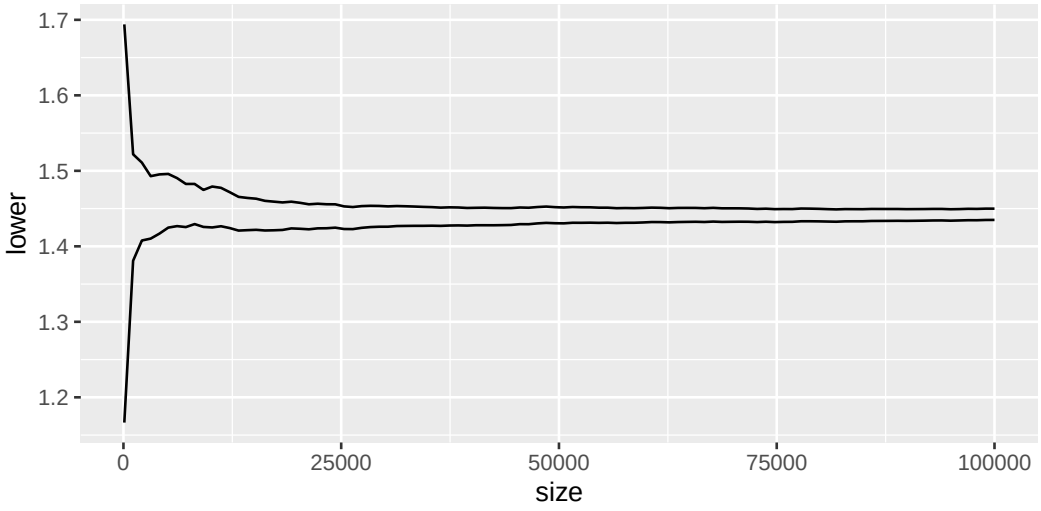
```
## [1] 1.442
```

To add a 99.9% confidence intervals:

```
## t.test will do other things, we only need the conf.int
t.test(hX, conf.level = 0.999)$conf.int
```

```
## [1] 1.435 1.450
## attr(,"conf.level")
## [1] 0.999
```

Confidence Intervals with Larger Samples



Probabilities and CDFs

Probability as an Integral

Many statistical tasks can be viewed as taking an expectation for a suitable function g.

In particular, if g is an **indicator function**, then the expectation of g is a **probability**.

Indicator functions:

$$\mathbf{1}(s) = \begin{cases} 1 & : s \text{ is true} \\ 0 & : s \text{ is false} \end{cases}$$

Note: In R we get indicators "for free" by writing things like: `x <= 3`.

Expectation of Indicators

Suppose we have a **continuous random variable** X and g **be an indicator function**. For example, $g(X) = \mathbf{1}(X > 3)$.

Since $g(X)$ can be either 1 or 0 (it is a **random variable**), let A **be the event** (values of X) that $g(X)$ is 1.

Notice: A and A^c partition the sample space.

Claim: $E(g(X)) = P(A)$

Example: $E(\mathbf{1}(X > 3)) = P(X > 3)$

Proof

Remember: $g(X) = 1$ if A is true and $g(X) = 0$ if A^c is true ("not A").

$$\begin{aligned} E(g(X)) &= \int_{-\infty}^{\infty} g(x) f(x)\, dx \\ &= \int_A 1 \times f(x)\, dx + \int_{A^c} 0 \times f(x)\, dx \\ &= \int_A f(x)\, dx \\ &= P(A) \end{aligned}$$

Estimating a Cumulative Distribution Function

The CDF of a (continuous) random variable is defined as:

$$F(t) = P(X \leq t) = E(\mathbf{1}(X \leq t))$$

The **empirical CDF** is then the sample mean of $\mathbf{1}(X_i \leq t)$:

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(X_i \leq t)$$

This is the **first case** again: the density is already inside the integral, so we sample X directly and average — no multiply/divide needed.

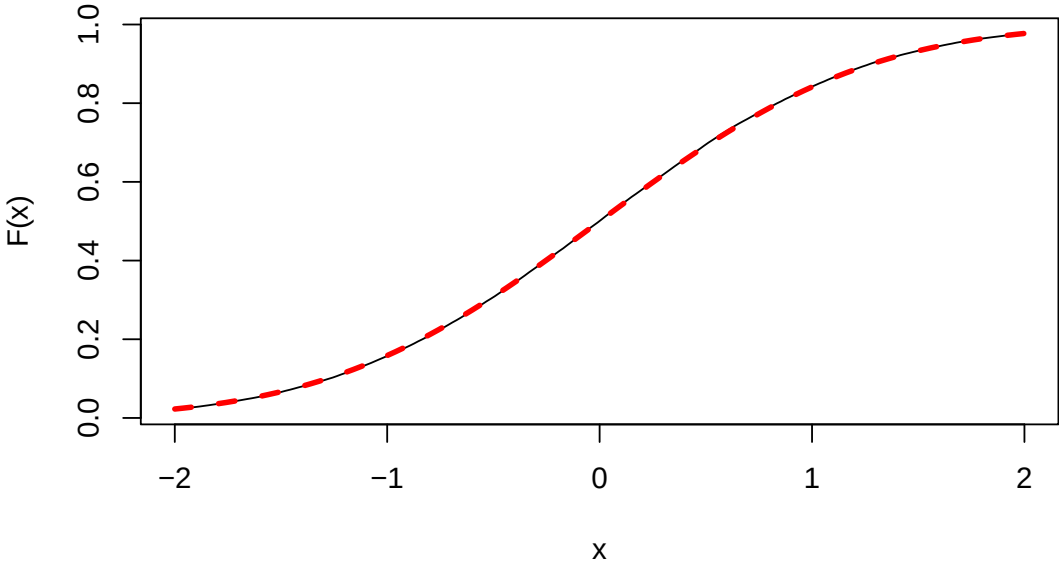
Example: Normal CDF

The CDF of the Normal distribution **does not have a closed form**.

For $X \sim N(0, 1)$, let's estimate $P(X \leq 1.96)$.

```
xs <- rnorm(1e5)
mean(xs <= 1.96)
```

```
## [1] 0.9752
```


F(x)
0.0
0.2
0.4
0.6
0.8
1.0
−2
−1
0
1
2
x

Better Confidence Intervals for Estimated Probabilities

The t-**distribution intervals** we used earlier would work when estimating a probability, but **we can do slightly better**.

Consider the random variable $W = \mathbf{1}(X \leq t)$. **What is this?**

- W can take values of 0 or 1
- $P(W = 1) = P(X \leq t) = \theta$, $P(W = 0) = 1 - \theta$.
- W is **Bernoulli**!

We have to estimate θ (our goal anyway), but this means we can use **confidence intervals for proportions** (i.e., `binom.test` instead of `t.test`).

```
binom.test(sum(xs <= 1.96), n = length(xs))$conf.int
```

```
## [1] 0.9742 0.9762
## attr(,"conf.level")
## [1] 0.95
```

Higher Dimensional Problems

Integrals in more than one dimension

Nothing in our recipe was one-dimensional. To integrate over a region of $\mathbb{R}^k$: sample a random **vector**, apply g, average.

The classic double integral — the area of the unit circle:

$$\pi = \int_{-1}^{1} \int_{-1}^{1} \mathbf{1}(x^2 + y^2 \leq 1)\, dx\, dy$$

Take $X, Y \overset{\text{iid}}{\sim} U(-1, 1)$, so the joint density is $f(x, y) = 1/4$ on the square. Multiply and divide:

$$\pi = 4 \int_{-1}^{1} \int_{-1}^{1} \mathbf{1}(x^2 + y^2 \leq 1)\, \tfrac{1}{4}\, dx\, dy = 4\, \mathsf{E}\left(\mathbf{1}(X^2 + Y^2 \leq 1)\right)$$

Throwing darts at a square

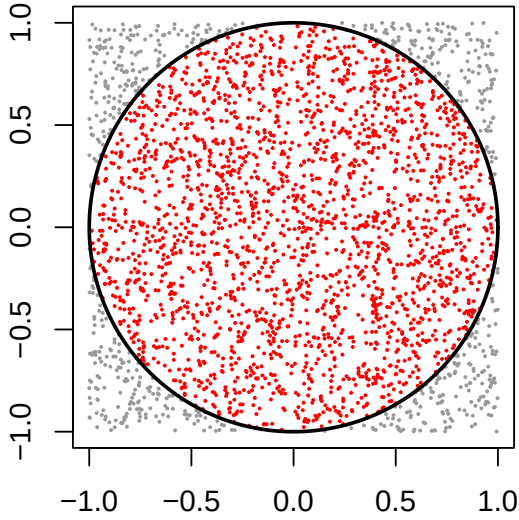
```
n <- 100000
x <- runif(n, -1, 1); y <- runif(n, -1, 1)
inside <- x^2 + y^2 <= 1
4 * mean(inside)
```

```
## [1] 3.143
```

```
4 * binom.test(sum(inside), n)$conf.int
```

```
## [1] 3.133 3.153
## attr(,"conf.level")
## [1] 0.95
```

An indicator again, so `binom.test` gives the confidence interval — everything from the last section carries straight over.


1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0

Aside: big-O notation

The CLT says the Monte Carlo error shrinks like $1/\sqrt{n}$. Is that fast?

To compare methods, we describe **how fast the error shrinks** as n grows:

$$\text{error} = O\left(g(n)\right)$$

means: for some constant C, $|\text{error}| \leq C\, g(n)$ for all large n.

The constant is deliberately ignored — big-O records the **rate**, which dominates once n is large.

A better rate always wins *eventually* — the question is how large n has to be first.

MC Error Rate $O(n^{-1/2})$

Recall the **standard deviation of a sample mean**:

$$\text{Var}\left(\hat{\theta}_n\right) = \text{Var}\left(\frac{1}{n}\sum_{i=1}^{n} g(X_i)\right) = \frac{1}{n^2}\sum_{i=1}^{n}\text{Var}\left(g(X_i)\right) = \frac{\tau^2}{n}, \quad \tau^2 = \text{Var}\left(g(X)\right)$$

so the typical error is

$$\sqrt{\tau^2/n} = \tau\, n^{-1/2}$$

The population variance τ^2 sets the **constant**; n sets the **rate**. So the error rate of Monte Carlo is $O(n^{-1/2})$. Notice that this **does not depend on** k – though the constant τ typically grows with k.

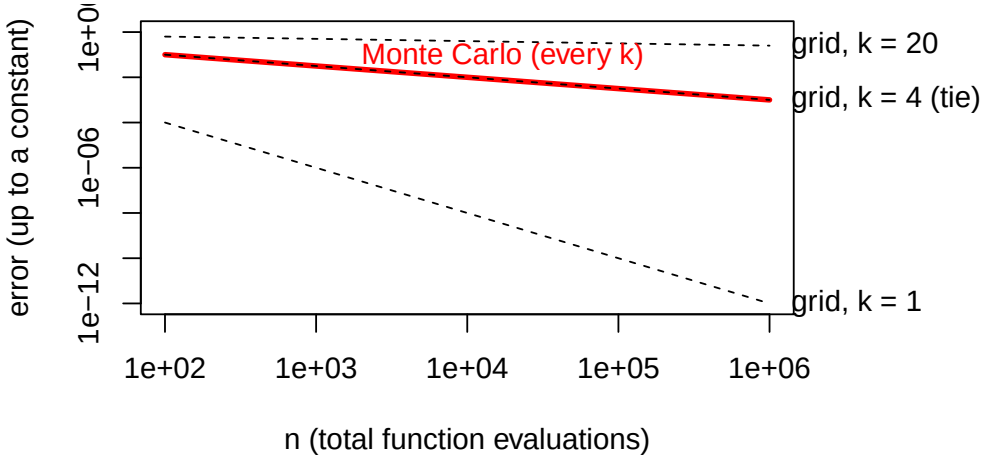
Grid search error rate $O(n^{-2/k})$

Take one cell of width d with midpoint m, and Taylor expand the integrand:

$$h(x) = h(m) + h'(m)(x - m) + \tfrac{1}{2}h''(\xi)(x - m)^2$$

- Integrating over the cell, the **linear term vanishes**: $(x - m)$ is symmetric about the midpoint.
- The leftover per cell is $\frac{1}{2}h'' \int_{\text{cell}}(x - m)^2\, dx \approx h''\, d^3/24$.
- Summing over the $(b - a)/d$ cells: total error $\approx (b - a)\, \overline{h''}\, d^2/24 = O(d^2)$.
- With n points, $d = (b - a)/n$, so the error is $O(n^{-2})$.
- In k dimensions, n points make the cells $d \propto n^{-1/k}$ wide: error $= O(d^2) = O(n^{-2/k})$.

Comparison using larger samples, more grid points.



Most real world problems are high dimensional (like linear regression). For this class we will tend to work with simpler examples.

Putting it to the test: a high-dimensional CDF

We just estimated the Normal CDF in one dimension. Now the **joint CDF** of $Z_1, \ldots, Z_k \overset{\text{iid}}{\sim} N(0,1)$, evaluated at $(1, \ldots, 1)$:

$$\theta_k = F(1, \ldots, 1) = P\left(Z_1 \le 1, \ldots, Z_k \le 1\right) = \int_{-\infty}^{1} \cdots \int_{-\infty}^{1} \prod_{j=1}^{k} \phi(z_j)\, dz = \Phi(1)^k$$

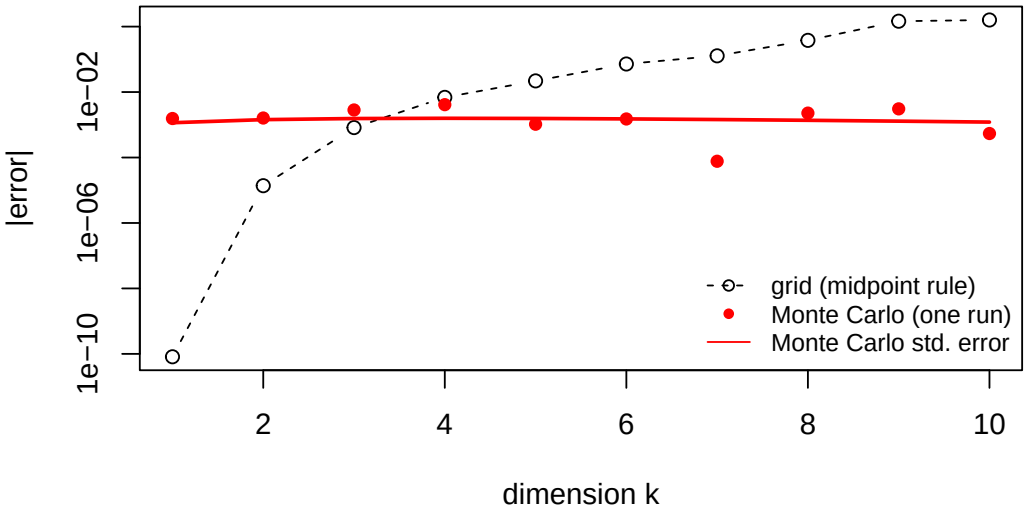
The exact answer is known, so we can score both methods. **Same budget for both**: 10^5 function evaluations.

- Grid: needs a *bounded* box first — chop at -8, where $\Phi(-8) \approx 6 \times 10^{-16}$. Then $m = \lfloor (10^5)^{1/k} \rfloor$ midpoints per axis on $[-8, 1]^k$.
- Monte Carlo: 10^5 draws of $Z \in \mathbb{R}^k$ — no box needed.

(Because the integrand factors, the k-dimensional midpoint value is just the 1-D midpoint value to the k-th power — we can *score* the full grid without enumerating it.)

The code

```
exact1 <- pnorm(1)  # one axis: P(Z <= 1)
n <- 1e5
grid_err <- map_dbl(1:10, function(k) {
  m <- floor(n^(1 / k) + 1e-9)                # points per axis
  mids <- seq(-8 + 4.5/m, 1 - 4.5/m, length.out = m)
  I1 <- sum(dnorm(mids)) * (9 / m)            # 1-D midpoint rule
  abs(I1^k - exact1^k)                        # grid value is I1^k
})
mc_err <- map_dbl(1:10, function(k) {
  zs <- matrix(rnorm(n * k), ncol = k)
  abs(mean(rowSums(zs <= 1) == k) - exact1^k)
})
```


|error|
1e−02
1e−06
1e−10
2
4
6
8
10
dimension k
grid (midpoint rule)
Monte Carlo (one run)
Monte Carlo std. error

Monte Carlo Integration: Summary

- Monte Carlo turns an integral into a **sampling problem**.
- To estimate $\int_a^b g(x)\,dx$: pick a density f positive on $[a, b]$, sample $X_i \overset{\text{iid}}{\sim} f$, average $g(X_i)/f(X_i)$.
- If the target is already $\mathrm{E}\,(g(X))$: sample and average, no ratio needed.
- R functions for RVs: `rsamples,dis the density (p,q‘` for CDF and quantiles).
- The sample mean is **unbiased** and **consistent** (LLN); the CLT gives **confidence intervals** using `t.test`.
- Probabilities are expectations of **indicators**, but use `binom.test` for CI.
- Multiple dimensions: sample a random **vector**, apply g, average.
- Error is $O(n^{-1/2})$ **in every dimension**; grids are $O(n^{-2/k})$, poor for large k.